The SOLID Design Principles

Sample study notes written for StudyBuddy. Free to share.

Why design principles matter

Most of the cost of software comes after its first release. Requirements change, bugs are fixed and features are added, and every change risks breaking something that already works. Design principles are rules of thumb for arranging classes so that change stays cheap: each change touches few classes, and the classes it touches are easy to understand and test.

Two ideas sit underneath all five SOLID principles. Cohesion describes how closely the parts of one class belong together; a cohesive class does one job and all of its methods serve that job. Coupling describes how much one class depends on the details of another; loosely coupled classes can change independently. Good designs aim for high cohesion and low coupling.

The name SOLID is an acronym for five principles: Single responsibility, Open-closed, Liskov substitution, Interface segregation and Dependency inversion.

S: The Single Responsibility Principle

A class should have only one reason to change. A reason to change usually corresponds to one group of people who might ask for a change, such as accountants, database administrators or the people who design reports.

Consider a class called Invoice that calculates its total, saves itself to a database and prints itself as a PDF. Three different groups could ask for changes to it: a new tax rule, a new database schema and a new report layout all force edits to the same class. Each edit can break the other two jobs, and the class is hard to test because testing the tax rule also needs a database.

The fix is to split the jobs. Invoice keeps the business logic, InvoiceRepository handles saving and loading, and InvoicePrinter handles the layout. Each class now has one reason to change, and each can be tested on its own.

A useful warning sign is a class whose description needs the word "and": "this class calculates totals and stores invoices and prints them".

O: The Open-Closed Principle

Software entities should be open for extension but closed for modification. In practice this means you should be able to add new behaviour by writing new code, not by editing code that already works and is already tested.

The classic violation is a long chain of if or switch statements on a type code:

```
double area(Shape s) {
    if (s.type == CIRCLE) return Math.PI * s.r * s.r;
    if (s.type == SQUARE) return s.side * s.side;
    // every new shape means editing this method
}
```

Every new shape forces an edit to this method and to every other method with the same switch. The fix is to put the varying behaviour behind an abstraction. A Shape interface declares area(), and Circle and Square implement it. Adding a Triangle means adding one new class; no existing class changes.

The Strategy and Template Method design patterns are two common ways of following this principle. The key skill is predicting which parts of a system are likely to vary and putting an abstraction exactly there, rather than everywhere.

L: The Liskov Substitution Principle

Objects of a subclass must be usable wherever objects of the base class are expected, without the program behaving incorrectly. A subclass may add behaviour, but it must keep the promises made by the base class.

Those promises are about behaviour, not only method signatures. A subclass must not demand more from its callers (stronger preconditions), must not deliver less (weaker postconditions) and must keep the invariants of the base class.

The well-known counterexample is a Square class that extends Rectangle. A Rectangle lets callers set the width and height independently. A Square must keep both sides equal, so setting the width also changes the height. Code written for rectangles, such as a test that sets the width to 5 and the height to 4 and expects an area of 20, fails when it receives a Square. Square is a rectangle in geometry, but not a valid subtype of this Rectangle class.

Signs of a violation include subclasses that throw UnsupportedOperationException for inherited methods, and client code that checks the concrete type with instanceof before calling a method.

I: The Interface Segregation Principle

Clients should not be forced to depend on methods they do not use. Large "fat" interfaces should be split into smaller, role-based interfaces.

Imagine a Worker interface with the methods work(), eat() and sleep(). A RobotWorker class has to implement eat() and sleep() even though they mean nothing for a robot, usually with empty bodies or exceptions. Every class that uses Worker also has to be recompiled when any of the three methods changes.

Splitting the interface into Workable, Feedable and Restable lets each class implement only the roles it really has, and lets each client depend only on the role it needs. Small interfaces are also easier to replace with test doubles.

D: The Dependency Inversion Principle

High-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.

A high-level module holds the important policy of the application, such as "send a reminder when an assignment is due". A low-level module holds a detail, such as "send an email through a particular mail server". If ReminderService creates a GmailSender directly, the policy is tied to one detail: switching to text messages, or testing without sending real email, requires editing ReminderService.

The fix is to let ReminderService depend on a MessageSender interface that it owns, and to have GmailSender and SmsSender implement that interface. The concrete sender is passed in from outside, usually through the constructor. This is called dependency injection.

Dependency inversion makes code easier to test. In a unit test, ReminderService can receive a fake MessageSender that only records the messages it was asked to send, so the test runs quickly, needs no network and can check exactly what would have been sent.

Summary

Principle	One-line rule
Single responsibility	One reason to change
Open-closed	Extend without editing
Liskov substitution	Subtypes keep the base class's promises
Interface segregation	No unused methods forced on clients
Dependency inversion	Depend on abstractions

The principles work together. Splitting responsibilities produces small classes, small classes need small interfaces, and depending on those interfaces makes it possible to extend and test the system without modifying it.